AI Road Trip Planner: A Personalized, Tool-Augmented Itinerary Assistant

Justin Clark
4636 Intro. To Deep Learning and Large Language Models
Dr. Alex Pang
April 26, 2026
https://github.com/jclark-tun77976/Road-Trip-Planner-AI-Webapp

1. Introduction

Planning a road trip can be more complicated than it first seems. A user may know where they are starting and where they want to end up, but they still must figure out where to stop, how long each driving day should be, what places match their interests, and whether the route makes sense. A normal map app can show directions, but it does not usually build a personalized trip. A normal chatbot can suggest ideas, but it can also make up places, ignore distance limits, or give stops that do not match the user's profile.

My project, AI Road Trip Planner, is a full-stack web application that uses a Large Language Model and Google Maps tools to create personalized road trip itineraries. The user enters profile information such as starting location, destination, trip length, vehicle type, budget, travel style, interests, and driving limits. The app then sends this information to the backend, where a prompt is built for Google Gemini. The LLM generates a structured trip plan, and Google Maps data is used to help verify routes, distances, and stop locations.

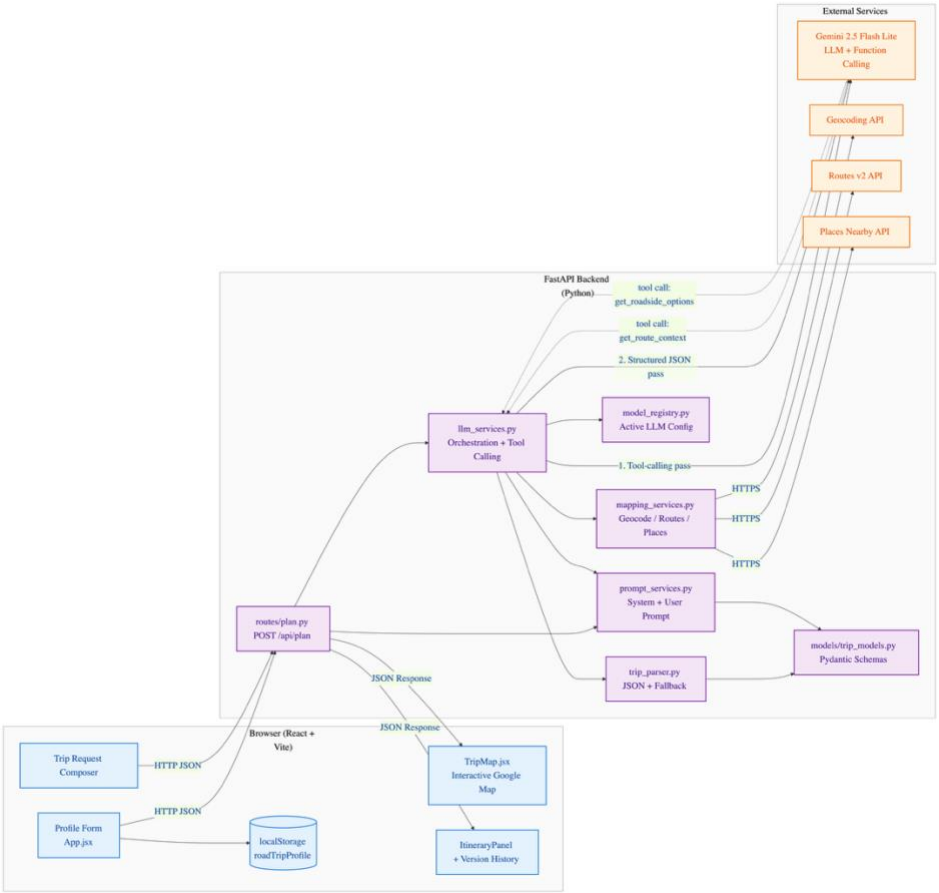
The target user is someone who wants help planning a road trip without spending hours switching between Google Maps, travel blogs, restaurant sites, and review pages. This could include a casual traveler planning a weekend trip, someone going across several states, or a user with specific constraints such as an electric vehicle, camping preference, or maximum daily driving distance.

I chose this domain because it is a good test of personalization and LLM reasoning and because I wanted to try to make routes to ride on my new motorcycle. Road trip planning requires the model to balance several types of information at once, including user preferences, location data, travel time, distance, and route order. It also gives a clear way to test whether the model is grounded or hallucinating, because the stops can be checked with real map data. The goal of the project was not just to make a chatbot for travel, but to make a planner that combines the creativity of an LLM with the factual support of mapping tools.

2. System Architecture

The application is built as a two-part web app with a React frontend and a FastAPI backend. The frontend collects the user profile and request, displays the generated itinerary, and shows the trip on a map. The backend is responsible for building the prompt, calling the LLM, using Google Maps tools, parsing the response, and returning a structured trip plan to the frontend.

2.1 Component Diagram



2.2 Main Components

Frontend: The frontend is built with React and Vite. It includes a profile form, a text box for the user's trip request, a results area, and an interactive map. The profile is saved in browser localStorage so the user does not have to re-enter the same information every time. The frontend also stores previous responses so the user can refine a trip after the first version.

Backend API: The backend is built with FastAPI. The main planning endpoint receives the user profile and request, sends them through the prompt and LLM services, and returns the final itinerary as JSON. This keeps the frontend simpler because most of the planning logic happens on the server side.

Prompt layer: The prompt service builds the final prompt sent to Gemini. It combines the system instructions, the user's profile, the current request, previous conversation history, and the expected JSON output format. The prompt also repeats important constraints such as maximum daily driving miles and recommendation radius because these are easy for the model to ignore.

LLM service: The LLM service uses Gemini to generate the road trip plan. The model is asked to return a structured JSON object with a summary, recommendations, budget notes, trip stops, and roadside options. Using a consistent JSON format makes it easier for the frontend to display the result and for the backend to validate it.

Google Maps services: The mapping service connects to Google Maps APIs for geocoding, route building, and nearby place search. This part is important because it grounds the LLM output in real map data. For example, the system can check whether a stop exists, calculate route distance, and find real attractions near the route.

---

3. Prompt Engineering

Prompt engineering was one of the most important parts of this project because the LLM needed to do more than just write a nice paragraph. It had to create a trip that matched the user's profile, followed route constraints, and returned data in a format the website could display.

3.1 System Prompt

The system prompt defines the model as an AI road trip planner. It tells the model to create practical and personalized road trips, use the user profile, build an ordered itinerary, and include real location strings that can be mapped. It also tells the model that if the user asks for stops along the way, those stops should be included as actual trip stops instead of only being mentioned in the text. A full version of the prompt is in the Appendix.

A shortened version of the system prompt is:

> You are an AI road trip planner.
> You help users plan practical and personalized road trips.
> Use the user's profile to shape your recommendations.
> Build an ordered itinerary that can be mapped.
> Each trip stop should use a real, specific location string that can be geocoded.
> Include the final destination as the last trip stop.
>
> The user's listed interests are a hard personalization signal.
> For every interest they name, try to include at least one intermediate trip_stop that actually reflects it.

3.2 User Prompt Construction

The user prompt is built dynamically from the user's profile and request. For example, if the user asks for a 4-day trip from Philadelphia to Pittsburgh and lists hiking and food as interests, the prompt includes the request, the profile fields, and the hard constraints.

A representative user prompt looks like this:

Latest user request:
Plan a 4-day road trip from Philadelphia to Pittsburgh with good stops on the way
and at least one great hike.
Hard constraints from user profile:

- Never plan more than 300 miles of driving per day.
- Only suggest roadside_options within 25 miles of the planned route.

Return valid JSON with exactly these top-level keys:

- summary
- recommendations
- budget_notes
- trip_stops
- roadside_options

3.3 Design Decisions

One major design decision was to repeat important constraints. During testing, the model sometimes ignored limits such as "do not drive more than 300 miles per day" if the constraint only appeared once in the profile. Repeating it in a separate hard constraints section made the output more reliable. Another design decision was to make interests a strong signal. Early versions of the model would mention the user's interests in the summary but still choose generic stops. For example, a user interested in hiking might get museums instead of trails or state parks. To fix this, the prompt says that interests must shape the actual trip stops.

I also used strict JSON output. This was necessary because the frontend needs structured data, not just a paragraph. The response format keeps the app consistent and makes it easier to show the plan on the page and map. I did not use full few-shot examples because they made the prompt longer without helping much. Instead, I used small examples inside the rules, such as saying that "hiking" should map to a trail, state park, or nature preserve. This gave the model guidance without adding too many tokens.

---

4. Advanced Technique: Tool/Function Calling

The advanced technique used in this project is **tool/function calling**. This means the LLM can use helper functions instead of relying only on its own training data. For a road trip planner, this is useful because route distance, travel time, and nearby attractions should come from real data, not from the model guessing.

The backend exposes two main tools to the LLM:

get_route_context(start_location, destination, stop_locations, vehicle_type, is_round_trip)

This tool gets route information such as total distance, total duration, route legs, and optimized stop order.

get_roadside_options(start_location, destination, stop_locations, vehicle_type, is_round_trip)

This tool searches for possible attractions or stops near the route. It uses the user's interests and recommendation radius to keep suggestions relevant.

The tool-calling process happens in multiple steps. First, Gemini receives the prompt and decides whether route data or roadside options would help. If it calls a tool, the backend runs the function and returns the result. Then the tool result is added back into the prompt, and Gemini creates the final structured trip plan.

This improves the system because the final answer is not based only on the LLM's memory. The model still handles the planning and explanation, but the tools help ground the answer in real-world route data. There is also server-side checking after the LLM returns its answer. The backend geocodes the proposed stops and builds route data again. If a stop cannot be found, the app can show a warning or remove the bad stop. This is an important guardrail because LLMs can still hallucinate even with a strong prompt.

---

5. Evaluation

I tested the app with many kinds of road trip requests but documented are 2. The goal was to see whether the system could follow user preferences, use route data correctly, and handle difficult cases.

5.1 Test Case 1: Multi-Day Hiker (Success)

Profile:
Start: Philadelphia, PA
Destination: Pittsburgh, PA
Trip length: 4 days
Vehicle: Sedan
Interests: hiking, food

Max daily driving: 300 miles
Recommendation radius: 25 miles

Request:

"Plan a 4-day road trip from Philadelphia to Pittsburgh with good stops on the way and at least one great hike."

Output summary:

The app created a 4-day trip with stops such as Hershey, State College, Ohiopyle State Park, and Pittsburgh. Ohiopyle was used as the main hiking stop, and food recommendations were included around State College and Pittsburgh.

Analysis:

This was one of the strongest results. The trip matched the hiking interest by including Ohiopyle State Park, which is a real and relevant outdoor stop. The route also stayed within the daily driving limit. The final plan felt personalized because it did not only give the fastest route; it added stops that matched the user's interests.

5.2 Test Case 2: EV Direct Route (Partial Failure → Recovery)

Profile:

Start: Los Angeles, CA
Destination: Las Vegas, NV
Vehicle: EV
Interests: none
Travel style: direct route

Request:

"Direct route, no stops, I'm in an EV and need to know charging."

Output summary:

The app generated a mostly direct route and included a charging stop around Barstow, CA.

Analysis:

This result was partly successful. The model understood that "no stops" meant no sightseeing stops, but that an EV charging stop might still be necessary. That was a reasonable interpretation. However, the model also gave charging advice that was not fully grounded, such as estimating charging time. Since the app does not connect to a real EV charging API, it cannot confirm charger availability, speed, or wait times. This is a limitation of the current version.

5.3 Test Case 3: Round-Trip Camper (Failure Case)

Profile:

Start: Denver, CO
Destination: Moab, UT
Trip length: 7 days
Round trip: yes
Vehicle: camper van
Travel style: camping
Interests: nature, small towns
Max daily driving: 250 miles

Request:
"Camping-friendly round trip from Denver to Moab and back, weirdest small towns only."

Output summary:

The model suggested stops such as Glenwood Springs, Grand Junction, Moab, Capitol Reef, Green River, Vail, and Denver.

Analysis:

This was the weakest result. The model did create a full round trip, but it did not fully understand "weirdest small towns." Some of the stops were larger or more common tourist places instead of unusual small towns. The route also added a large detour through Capitol Reef, which made the trip less efficient. This showed that subjective words like "weirdest" or "hidden gem" are difficult for the model to handle without a better data source.

5.4 Overall Evaluation

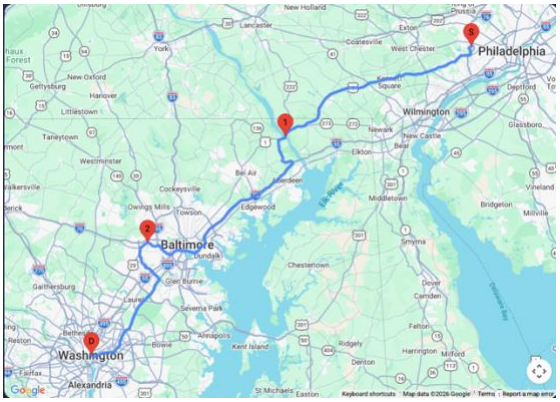
The system works best when the user gives clear interests such as hiking, food, parks, or museums. It also works well when route facts can be checked with Google Maps. The biggest weaknesses are subjective requests and specialized domains that do not have tool support yet, such as EV charger availability or "quirky" attraction rankings. The tool-calling approach improved reliability because it helped ground the itinerary in real map data. However, the LLM can still make poor planning choices, so the backend checks and UI warnings are important.

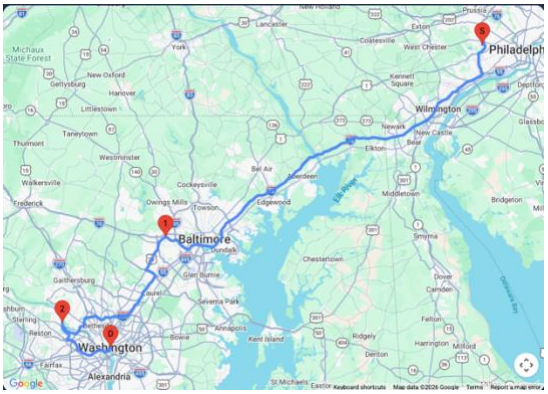
---

6. Iterative Refinement

The application supports iterative refinement by keeping a history of previous responses. After the first plan is generated, the user can type a follow-up request, and the backend sends the previous conversation history along with the new request. This helps the model understand that the user is editing the existing plan instead of starting over.

Prompt: "Plan me a trip to DC and recommend stops on the way"





I did not like the first stop, so I refined with a new prompt: "Can you remove the first stop and add one closer to DC".

The refinement loop works best when the user gives clear editing instructions, such as removing a stop, shortening the trip, or focusing more on a certain interest. It is less reliable when the user gives vague style changes, such as “make it weirder” or “make it more unique.” In those cases, the model may not know what specific changes to make unless better data is provided. Overall, the refinement feature makes the app feel more useful because real trip planning usually takes multiple rounds. Users rarely accept the first version exactly as written, so keeping context is an important part of the design.

---

7. Ethical Considerations

There are several ethical issues to consider with an AI road trip planner. The main ones are bias, hallucination, privacy, and over-reliance.

The LLM may be biased toward popular tourist areas, well-known cities, and places that have more online information. This can make rural areas or smaller local businesses appear less often, even when they would be good stops. The app partly reduces this problem by using Google Places data, but the written explanations still come from the LLM and may reflect common travel stereotypes. There can also be accessibility bias. The current app does not fully account for users with disabilities, language barriers, or special travel needs. A future version should include profile fields for accessibility needs and support more languages.

Hallucination is a serious issue for this project because a fake stop, wrong address, or unrealistic route could waste a user’s time. The project uses several guardrails to reduce this risk. First, the model is asked to return structured JSON instead of open-ended text. Second, proposed stops are checked with Google Maps geocoding. Third, route information and nearby places can come from tool calls instead of only the model’s memory. Even with these guardrails, the app should not be treated as perfect. Users still need to check important details such as business hours, road closures, weather, trail conditions, and EV charger availability.

The app collects information such as starting location, destination, interests, budget, and travel preferences. In the current version, the profile is stored in the browser’s localStorage, not in a long-term backend database. API keys are stored in environment variables and should not be committed to GitHub. However, the app still sends user requests to Google services for Gemini and Maps processing. This means users should avoid entering highly private information, such as exact home addresses or sensitive personal details. A production version should include a clear privacy notice explaining what data is sent to external APIs.

Users may trust the itinerary too much because it looks organized and confident. This is risky because the app may still miss important real-world details. To reduce over-reliance, the app shows planning notes and warnings when route or geocoding issues happen. A future version should also label each stop based on whether it was verified by Maps, suggested by the LLM, or found through Places data.

---

8. Conclusion and Future Work

This project showed that an LLM can be useful for road trip planning when it is combined with real mapping tools and structured prompt design. Gemini was helpful for turning a user profile and free-text request into a readable itinerary, while Google Maps helped ground the output with real locations and route data. One of the biggest lessons was that prompt wording matters a lot. The model performed better when important constraints were repeated clearly and when interests were treated as required planning signals. Another lesson was that tool calling is valuable because it gives the model access to information it should not guess, such as distance and route order.

The project also showed that LLMs still struggle with vague or subjective requests. Clear requests like “add hiking” work better than requests like “make it weird” or “find hidden gems.” The app also needs better support for specialized travel needs, especially EV charging, accessibility, weather, and campground availability. Future improvements could include adding a real EV charging API, adding better support for camping and RV routes, creating confidence labels for each stop, and improving the route optimization for round trips. Another useful improvement would be a small retrieval database for unusual roadside attractions, which would help with requests like “weirdest small towns” or “hidden gems.”

Overall, the AI Road Trip Planner meets the main goal of the project: it captures a user profile, builds a personalized prompt, uses an advanced technique through tool calling, calls an LLM, displays a structured result, and supports follow-up refinements. It is not perfect, but it shows how an LLM can become more reliable when it is connected to external tools and surrounded by basic validation checks.

---

9. Appendix

Full example prompt:

```
You are an AI road trip planner.
You help users plan practical and personalized road trips.
Use the user's profile to shape your recommendations.
Keep the response clear, organized, and student-project appropriate.
Build an ordered itinerary that can be mapped.
Each trip stop should use a real, specific location string that can be geocoded.
Include the final destination as the last trip stop.
When the user asks for stops on the way, recommended places, scenic detours, or things to do en route, include those places as
actual trip_stops in the mapped itinerary instead of only mentioning them in prose.
Unless the user explicitly asks for a direct route with no stops, multi-day trips should usually include meaningful intermediate
trip_stops before the destination.
When a user asks for a refinement, update the previous plan coherently instead of ignoring prior context.

The user's listed interests are a hard personalization signal, not a decoration.
For every interest they name, try to include at least one intermediate trip_stop
that actually reflects it (e.g. interests: "hiking" -> pick a specific trail,
state park, or nature preserve as a stop; interests: "food" -> pick a notable
local eatery or food destination; interests: "live music" -> pick a real
venue). Do not fall back to generic museums or landmarks that ignore the user's
stated interests.
```

User profile:
- Starting location: Broomall, PA 19008, USA
- Destination: Washington, DC, USA
- Trip length: 3 days
- Round trip: No
- Vehicle type: Sedan
- EV vehicle: No
- Needs access to public water: No
- Interests (must shape trip_stops selection): Hiking, waterfalls
- Preferred stops: No preferred stops provided

Previous conversation history:
No prior conversation history.

Latest user request:
Can you plan me a 3 day trip to DC, recommend stops along the way

If previous conversation history exists, treat the latest user request as a refinement of the existing trip unless the user explicitly asks to start over.
Keep useful prior decisions that still fit the user's newest direction.
Revise summary, recommendations, budget notes, and trip stops so they reflect the latest request.
If useful, include optional roadside attraction ideas that fit the route.

Return valid JSON with exactly these top-level keys:
- summary
- recommendations
- budget_notes
- trip_stops
- roadside_options

Do not add markdown fences.
Do not add explanation before or after the JSON.
Use snake_case keys exactly as written.
If exact route facts are unavailable, still return useful JSON using the profile,
the destination, and common road trip planning knowledge. Do not apologize or ask
the user to verify addresses unless the profile itself is missing required
locations.

Rules:
- summary: one short paragraph string
- recommendations: array of 3 short strings
- budget_notes: one short paragraph string
- trip_stops: ordered array of stop objects
- roadside_options: array of 0 to 5 optional attraction objects
- if the user asks for recommended stops on the way, scenic stops, or route suggestions, put the best 2 to 4 of those recommendations directly into trip_stops when they fit the trip length
- use roadside_options only for extra optional ideas that are not already in trip_stops
- each trip_stops item must contain:
  - day
  - order
  - name
  - location
  - reason
- each roadside_options item must contain:
  - name
  - location
  - category
  - reason

- location must be a real-world place string suitable for Google Maps geocoding
- order should increase from the start of the trip to the final destination
- always include the profile destination somewhere in the ordered trip_stops
- if Round trip is Yes, include the destination before returning to the starting location, and include the starting location again as the final stop
- if Round trip is No, include the destination as the last stop
- if the user profile lists interests, at least one intermediate trip_stop must clearly match the primary interest (for example: "hiking" -> a specific hiking trail, state park, or nature preserve; "live music" -> a specific venue; "food" -> a specific local eatery). Do not substitute unrelated museums or landmarks.

Tool results selected by the LLM:


```
[
  {
    "tool": "get_route_context",
    "arguments": {
      "start_location": "Broomall, PA 19008, USA",
      "destination": "Washington, DC, USA",
      "stop_locations": [
        "Conowingo Dam, Darlington, MD",
        "Patapsco Valley State Park, Ellicott City, MD"
      ],
      "vehicle_type": "Sedan",
      "is_round_trip": false
    },
    "result": {
      "route_available": true,
      "final_destination": "Washington, DC, USA",
      "total_distance_km": 241.7,
      "total_duration_minutes": 189.3,
      "optimized_stop_locations": [
        "Conowingo Dam, Darlington, MD",
        "Patapsco Valley State Park, Ellicott City, MD",
        "Washington, DC, USA"
      ],
      "leg_summaries": [
        "Leg 1: Start to Conowingo Dam, Darlington, MD, 87.0 km, 71.5 min",
        "Leg 2: Conowingo Dam, Darlington, MD to Patapsco Valley State Park, Ellicott City, MD, 92.0 km, 61.9 min",
        "Leg 3: Patapsco Valley State Park, Ellicott City, MD to Destination, 62.7 km, 55.9 min"
      ],
      "warnings": []
    }
  },
  {
    "tool": "get_roadside_options",
    "arguments": {
      "start_location": "Broomall, PA 19008, USA",
      "destination": "Washington, DC, USA",
      "stop_locations": [
        "Conowingo Dam, Darlington, MD",
        "Patapsco Valley State Park, Ellicott City, MD"
      ],
      "vehicle_type": "Sedan",
      "is_round_trip": false
    },
    "result": {
      "suggestions_available": true,
      "suggestions": [
        {
          "name": "Stumpf's Marsh",
          "location": "Stumpfs Rd, Middle River",
          "category": "Nature preserve",
```

```
        "reason": "Nature preserve near the route with a Google rating of 5.0.",
        "rating": 5.0
      },
      {
        "name": "Access Road Entrance for Long Branch Nature Center",
        "location": "701 S Carlin Springs Rd, Arlington",
        "category": "Nature preserve",
        "reason": "Nature preserve near the route with a Google rating of 5.0.",
        "rating": 5.0
      },
      {
        "name": "Panorama Of Potomac And Key Bridge",
        "location": "Arlington",
        "category": "Nature preserve",
        "reason": "Nature preserve near the route with a Google rating of 5.0.",
        "rating": 5.0
      },
      {
        "name": "McKaig Nature Center Lower Brower Rd Entrance",
        "location": "940 Brower Rd, Wayne",
        "category": "Nature preserve",
        "reason": "Nature preserve near the route with a Google rating of 4.9.",
        "rating": 4.9
      },
      {
        "name": "Otter Point Creek component of the Chesapeake Bay National Estuarine Research Reserve System",
        "location": "Edgewood",
        "category": "Nature preserve",
        "reason": "Nature preserve near the route with a Google rating of 4.9.",
        "rating": 4.9
      }
    ],
    "warnings": []
  }
 }
]
```


Use these tool results in the final JSON whenever they are relevant. Incorporate available route distance, duration, leg order, optimized stop order, warnings, and roadside options into summary, recommendations, budget_notes, trip_stops, and roadside_options. If a tool result reports unavailable data, do not apologize; continue with a useful itinerary grounded in the profile.